Clients Rot, Connections Don't: Degradation, Silent Failures and Refresh in Long-Lived WMI Connections

Author Name
Affiliation

Draft v3, 2026-10-20. **[Red: author input]**

Abstract

Windows management agents commonly recycle their WMI connections every few hours on the belief that long-lived connections degrade. We test that belief with a client whose COM usage is verified on every query, which checks results against non-WMI ground truth and attributes resource use to the provider host serving each query. On two hosts, one Windows 10-based and one Windows 11-based, one set of connections served 381,600 and 194,400 queries over 6 h at 100× and 50× a production agent's rate (about 25 and 12.5 production-days of queries). Latency, median and 99th percentile, did not change with connection age on either host. Client memory did not grow measurably on the first (0.039 MB/h, 95% CI [−0.007, 0.086]; an upper bound near 7.5 MB/year at the production rate); on the second it grew slowly (0.230 MB/h [0.190, 0.271]), well below our pre-set bound (about 40 MB/year at the production rate) and fully reclaimed when the connections closed, though whether it reaches a plateau is still open. The degradation we could reproduce was client-induced: an unreleased `VARIANT` array leaks about 140 B per read, and cross-apartment calls to the scripting objects cost about 3× the wall-clock time per collection cycle (5–8× the CPU). *Silent* failures matter more: a popular Go COM library ends enumeration without reporting the error, three of a production agent's 23 queries fail on every poll on both architectures while logged as successes, and `Win32_SystemDriver` omits a registered driver whose binary is missing, depending on the properties selected. Fault injection shows where refresh matters: on both hosts a restart of the WMI service broke every existing connection permanently; reconnecting on the error restored service within seconds, a 1-hour timer after 30 min, and a replica of the agent's pool not within the hour, because its health check does not recognise the error. A crash of a provider host cost at most the call it was serving; the connection survived. Agents should invest in client correctness and result verification, and reconnect on errors rather than on a clock.

1 Introduction

Windows Management Instrumentation (WMI) is the standard inventory and telemetry interface on Windows; endpoint agents, monitoring systems and configuration tools poll it continuously. A widespread practice is to treat WMI connections as perishable: agents tear down and recreate their `IWbemServices` connections on a timer or a query budget. The belief is documented in the tooling as well as in operations: a widely used Go WMI package warns of a "memory bug" and advises callers to run queries in a dedicated process and exit after a bounded number of queries [11], and reported client leaks have been traced to unreleased COM objects rather than to connection age [12]. The production agent we studied ("Agent A"; **[state author/vendor relationship to Agent A, and whether its source was available to us]**) recycles every 6 h or 50,000 queries, a rule motivated by observed memory growth, latency creep and timeouts.

Whether those symptoms come from *connection age* or from the *client* has, to our knowledge, not been measured. It is hard to measure: an incorrect client produces exactly these symptoms, the usual metric (memory summed over all `WmiPrvSE.exe` processes) mixes every WMI client on the host, and many WMI failures are silent.

We contribute (1) a method that removes these confounders: a self-tested correct client, per-provider-host attribution, an ETW meter of foreign WMI clients, and non-WMI ground truth (§3); (2) evidence on two architectures that a correct client's connections do not degrade in latency, and that client memory stays small and below a pre-set bound, at accelerated query counts (§4); (3) measurements of two client bugs that produce the folklore symptoms, and a catalogue of silent failures (§5, §6); and (4) a fault-injection comparison of refresh strategies with concrete guidance (§7, §8).

2 Background

A client calls `IWbemServices` in the WMI service (`Winmgmt`), which dispatches each class to a provider. In-process providers run in shared `WmiPrvSE.exe` host processes, one per hosting model such as NetworkService or LocalService [1]. WMI unloads a provider once it has been idle for its cache-control interval (`ClearAfter`; unloading takes "anywhere from the interval … to twice that interval") [2, 3], and enforces per-host quotas on memory, threads and handles [4]. Queries issued with the return-immediately and forward-only flags (0x30) are semisynchronous: `ExecQuery` returns an enumerator at once, and errors surface later from

IEnumWbemClassObject::Next, which returns WBEM_S_FALSE at the normal end of a result [5]. The scripting objects (SWbemLocator and relatives) are apartment-threaded: a call from another apartment is marshalled to the owning single-threaded apartment (STA) [6], and property values arrive as VARIANTs that the caller must free with VariantClear, which frees strings and arrays [7]. A client that walks a result set through the scripting objects enumerates an SWbemObjectSet via IEnumVARIANT::Next [18] rather than IEnumWbemClassObject::Next; the two share the convention that S_FALSE with no object marks the normal end of a result. Throughout, we distinguish the *outer* HRESULT a client sees at the call boundary from the *effective* error code: a failure dispatched through IDispatch::Invoke is reported to the caller as DISP_E_EXCEPTION with the originating WBEM or RPC code carried in the EXCEPINFO, so a health check that inspects only the outer code can miss a connection-level fault [17].

3 Method

A correct client. The probe (Go, go-ole) gives every connection its own OS thread initialised as an STA, and makes every COM call for that connection, including release, on that thread. Every VARIANT is cleared exactly once; a ledger counts acquisitions and releases per query and flags outstanding or double releases. Enumeration uses a strict loop: S_OK yields an item, S_FALSE with no item ends the result, and any other code is an error recorded with its HRESULT, WBEM code and the operation that failed. A query that exceeds its deadline is abandoned on the caller's side and its row is written when WMI eventually answers.

Workload. The catalogue reproduces Agent A's 23 production queries verbatim (processes, services, drivers, event logs, firewall, Defender, WMI subscriptions, scheduled tasks, Device Guard, TPM, BitLocker), each at its production rate. A query runs only where its namespace exists: on the Windows 10-based host 22 queries in 7 namespaces (BitLocker's namespace is absent; 636 queries/h), on the Windows 11-based host all 23 in 8 namespaces (648 queries/h). The probe holds three connections to root\cimv2 and one to each other namespace (9 and 10 connections). An acceleration factor k multiplies every rate. A pre-flight calibration on each host (10-minute steps, $k \in \{1,5,10,20,50,100\}$) measured probe-induced CPU (probe + all provider hosts + Winmgmt, as a percentage of the 2-vCPU host) of 0.4, 1.6, 3.3, 6.3, 15.6 and 29.9% on the Windows 10-based host and 0.6, 2.8, 5.5, 10.7, 27.3 and 53.7% on the Windows 11-based host, with no dropped queries; k_H, the largest k within a 40% budget, is 100 (the top of the ramp) and 50. The two hosts' virtual CPU models differ, so absolute latency and CPU are compared within a host, not across hosts.

Attribution and isolation. Every 10 s the probe samples itself, the Winmgmt service host (PID from the service control manager) and each WmiPrvSE by PID and creation time, labelled with its account and the provider DLLs it has loaded; this maps each queried class to the host serving it. Nothing here uses WMI. The class-to-host map was the same on both hosts: the CIMWin32, event-log, firewall, scheduled-task, Device Guard, TPM and (where present) BitLocker providers share the NetworkService host, and Defender's provider runs in the LocalService host. A real-time ETW session on Microsoft-Windows-WMI-Activity [8] counts operation starts by client PID, so foreign WMI clients are observed rather than assumed absent: in a 60-minute idle baseline it counted 3 and 15 foreign operations per hour on the two hosts, against a ceiling of 60/h fixed in the pre-registered design (one foreign operation per minute on average, under 0.2% of the probe's own queries at k_H; a window above it would have been flagged, and an arm with more than 10% flagged windows discarded). During the self-test and client-bug arms this session ran continuously; during the fault arms it was duty-cycled (§8), so foreign activity there is sampled. Because service-host grouping differs across editions and memory sizes [19], we read from the service control manager whether Winmgmt runs in its own svchost.exe or shares one: on both hosts it ran alone in its own. We found that tasklist.exe, the CIM cmdlets and PowerShell remoting all issue WMI calls; host inspection during runs was restricted to tools measured at zero WMI operations.

Ground truth. Results are compared with non-WMI sources taken immediately before and after each query: the process list (NtQuerySystemInformation), the service control manager, and invariants captured at the start. Five silent classes are counted: S1 impossible-empty, S2 truncated (a key present in both truth snapshots is missing), S3 enumeration ended by an error (what a lenient loop would report as success), S4 partial objects, S5 content mismatch.

Testbed and statistics. Two virtual machines with 2 vCPU each, one Windows 10-based (4 GB) and one Windows 11-based (8 GB), both with Defender real-time protection on and no other WMI-heavy software. Both hosts ran the same probe build and analysis code. Growth is the OLS slope of 10 s samples after a 30-minute warm-up with Newey–West standard errors (15-minute lag window) [10]. Pass criteria were fixed before the runs, in a timestamped commit of the design (7f3cbb9, 2026-10-16; the analysis implementing them, 3cfbca5, is unchanged since 2026-10-17) **[archive the repository publicly so these timestamps are verifiable]**: client private-byte slope < 1 MB/h and < 3 B per array read (upper 95% bound), handle range ≤ 50, and ≤ 5 MB retained after a 1-hour release test. Memory figures use MB $= 10^6$ bytes throughout.

Table 1: ST-1: one set of connections, strategy *none*, 6 h, on each host. All slopes are over the same 5.5-hour window (after the 30-minute warm-up); CIs are 95%, HAC. Memory in MB = 10^6 B.

	Win 10-based	Win 11-based
k (production-days of queries)	100 (25)	50 (12.5)
Queries	381,600	194,400
ok	360,000	183,598
deterministic error / abandoned	21,600 / 0	10,800 / 0
Silent failures above truth baseline	0	0
Client private bytes (MB/h)	0.039	0.230
	[−0.007, 0.086]	[0.190, 0.271]
per array read (B)	0.06	0.39
	[−0.01, 0.14]	[0.32, 0.45]
array reads	3.48 M	3.28 M
Client handles: range	17	9
Retained after 1 h release (MB)	−0.41	−0.90
NetworkService host (MB/h)	0.085	0.267
	[0.066, 0.103]	[0.142, 0.392]
LocalService host (MB/h)	0.036	0.090
	[0.001, 0.071]	[0.057, 0.122]
Winmgmt host (MB/h)	−0.020	0.034
	[−0.043, 0.003]	[0.016, 0.052]
Latency p99, last vs. first hour[a]	−1.1 to +0.4%	−2.2 to +0.8%
Recycles; induced CPU	0; 30.2%	0; 24.8%

[a] range over the query keys with median latency above 100 ms (six and seven keys).

4 A correct client's connections do not degrade

The self-test arm (strategy *none*: one set of connections held open for the whole run, with the probe's per-query COM ledger active) doubles as our longest correct-client run on each host (Table 1). At $k = 100$ its 6 hours carry the query count of about 25 days at the production rate (12.5 days at $k = 50$). This accelerates the *count* of queries, not the age of the connection and not the provider lifecycle: at these rates the providers never go idle long enough to unload, so the arm contains none of the unload/reload cycles a production client sees every few minutes (below). All client invariants held on every query on both hosts (correct apartment, no outstanding, double or leaked `VARIANT`s, no panics), and both passed the pre-set criteria.

Windows 10-based host. Client memory did not grow measurably. If growth tracks queries, we can rule out at the production rate a client-side leak larger than about 0.86 kB/h (roughly 7.5 MB/year) at 95% confidence. Median latency per query key stayed flat across connection age, from about 2 ms (repository classes) to 230 ms (`MSFT_NetFirewallRule`, 260 rows; first to last hour of the window −2%). The tails were as flat: for the six keys with medians above 100 ms, the 95th and 99th percentiles moved by at most 1.6% from the first to the last hour, no key's 99th percentile rose by more than 1.9 ms, and no successful query took longer than 363 ms against a 30 s deadline (none was abandoned). The `Winmgmt` host was flat (−0.020 MB/h [−0.043, 0.003]). The provider hosts grew mostly early: by thirds of the window, the LocalService host 0.19, 0.01 and −0.03 MB/h, and the NetworkService host, serving six providers, 0.13, 0.07 and 0.02 MB/h [−0.09, 0.13] (0.085 MB/h [0.066, 0.103] over the window, about 0.5 MB in 5.5 h). Six hours cannot distinguish a plateau from slow growth there; the 7-day run will. The single client handle step (339 to 347) coincides with one additional Go runtime thread and is flat before and after. A 7-day run at $k = 100$ on the Windows 10-based host (about 700 production-days of queries per connection, though not of provider reloads; see below) is in progress and will replace this section's age bound.

Windows 11-based host. Latency was again flat with connection age: first-to-last-hour medians moved by less than 1% for `MSFT_NetFirewallRule` (648 ms, 492 rows), `Win32_SystemDriver` (272 ms) and `Win32_Service` (191 ms), and by 2% for the full process list (167 to 170 ms). For the seven keys with medians above 100 ms the 95th and 99th percentiles moved by at most 2.6%, no key's 99th percentile rose by more than 3.2 ms, and no successful query took longer than 940 ms against a 30 s deadline (none was abandoned). The provider hosts and the `Winmgmt` host grew modestly over the window (Table 1), most of the rise in the first third: by thirds, the NetworkService host, now with seven providers, 1.37, −0.19 and −0.06 MB/h (30-minute medians 13.1 to 14.7 MB), the LocalService host 0.27, 0.09 and 0.02, and the `Winmgmt` host 0.13, 0.03 and 0.01 MB/h, the last two thirds' intervals all including zero. The client's private bytes, however, grew 0.230 MB/h [0.190, 0.271] (0.39 B per array read): distinguishable from zero, but 4.3× below our pre-set bound and small in absolute terms (27.5 to 29.0 MB over the arm, and about 4.6 kB/h, or 40 MB/year, at the production rate if it tracks queries). The growth was uneven across the window (by thirds 0.42, 0.04 and 0.40 MB/h), so these 6 hours cannot tell whether it continues linearly or approaches a plateau. The Go heap was flat (−0.010 MB/h [−0.041, 0.021]), locating the growth in native memory, and it was fully reclaimed when the connections were released: at the end of the release test the client was 0.90 MB below its post-warm-up level. This is consistent with memory held on behalf of open connections rather than an unbounded leak; the two are separable only by whether the growth plateaus, which a longer run on this host will decide (Figure 2). Because closing a connection reclaims this memory, the result can be read as an argument for periodic reconnection. It is a weak one. Even if the growth is linear in queries and never levels off, at the production rate it amounts to about 3.3 MB per month of connection age, so reconnecting monthly would bound it there; Agent A's 6-hour cycle reconnects about 120 times as often, and Section 7 shows that a timer recovers from the one fault that does break a connection only when it next fires (30 min in our runs), against seconds for reconnecting on the error. **[plateau run: a Windows 11-based strategy-*none* run of 48–72 h is proposed after the current battery; until then the paragraph above is the case]**

Providers reload anyway. On both hosts, read from the `__CacheControl` and `__ProviderHostQuotaConfiguration`

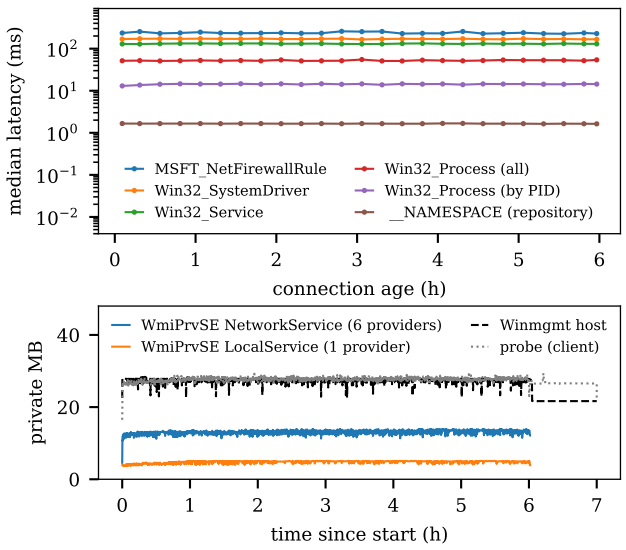


Figure 1: ST-1, Windows 10-based host. Top: median latency per 15-minute window against connection age for six query keys. Bottom: private bytes of the two provider hosts (labelled by account and number of providers; the providers are listed in Section 3), the Winmgmt host and the client; connections were released at 6 h. The Windows 11-based client is in Figure 2.

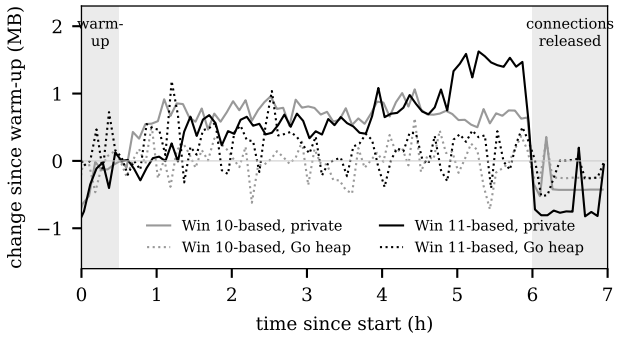


Figure 2: Client memory in ST-1 on both hosts: change in private bytes (solid) and Go heap (dotted) since the end of warm-up, 5-minute medians. Connections were released at 6 h.

instances, `ClearAfter` is 30 s (15 s for event sinks) and the provider-host quotas are identical (512 MB per host, 1 GB for all hosts, 4,096 handles, 256 threads, 32 processes): neither is tuned per architecture. At the production rate ($k = 1$) most classes are queried less often than that, and on the Windows 10-based host the WMI-Activity log showed six providers reloading every 1–2 minutes *inside* one long-lived host PID (event 5857 within 10 minutes); the Defender provider's host itself was replaced, and idle hosts exited 60–65 s after release (60 s on the Windows 11-based host). A production client thus gets a free refresh of provider state every few minutes; only its connection and the WMI service age.

5 What degrades is the client

Unreleased arrays (Agent A, issue 115). Property values read through `Properties_` that were never passed to `VariantClear` leak their `SAFEARRAY`s. In production this was steady memory growth (+1.45 MB/h over 13.5 h on a Windows 10 VM, +2.29 MB/h over 20 h on a laptop) that scaled with the firewall collector's frequency (+8.8 ± 0.6 MB/h at 60 s, +0.44 ± 0.52 after the fix) and that *recycling did not cure* (2.5 MB/h with recycling working): the leak is in the client. Our positive control re-enables the bug in the probe. On the Windows 10-based host at $k = 10$ for 1 h (3,180 queries and 31,650 array reads in the 30-minute measurement window) the client's private bytes grew 8.97 MB/h [8.82, 9.10], i.e. 141.5 B per array read (95% CI [139.3, 143.6]), against 0.06 B for the correct client (Table 1); the probe's ledger flagged 780 `VARIANT` values acquired and never released in the window (about 5.7 kB per retained value, consistent with the firewall-rule `SAFEARRAY`s). On the Windows 11-based host we ran the control at two rates, $k = 5$ and $k = 10$: per array read the leak was 139.4 B [135.8, 143.0] and 136.2 B [132.2, 140.1], the same at both rates and within 4% of the Windows 10-based value, while the hourly growth doubled (8.29 to 16.26 MB/h). The leak therefore tracks reads, not connection age, in our own runs as well as in Agent A's frequency sweep, and recycling did not release it.

Cross-apartment calls (issue 118). Agent A called the apartment-threaded scripting objects from multi-threaded-apartment goroutines, so COM marshalled every call into a single host STA (confirmed in debugger stacks). Moving each connection to its own STA cut the wall-clock time per collection cycle about 3× (process poll 1.31–1.40 s to 0.42–0.44 s; firewall rules 8.5 s to 2.8 s) and steady-state CPU on a busy 16-CPU laptop from 28.3% to 3.5–3.9% of a core (a 5–8× reduction), eliminating 1,490 abandoned calls in 69 h (≈21/h). On a quiet 2-vCPU VM the same bug cost 5.0% vs 0.9% and caused no abandoned calls: the symptom scales with host load, which makes it look like ageing on busy machines. This cost is specific to clients that drive the apartment-threaded scripting objects across apartments; native `IWbemServices` clients and clients on the newer Management Infrastructure (MI) API [16] do not marshal through a single STA and would not show it.

6 Silent failures

A library that swallows errors. In `go-ole` v1.3.0 [9], `oleutil.ForEach` (`oleutil.go`, lines 118–126) enumerates an object's `_NewEnum` through `IEnumVARIANT` and loops while the number of items returned is positive:

```
118  for item, length, err := enum.Next(1); length > 0;
119      item, length, err = enum.Next(1) {
120    if err != nil { return err }
121    ...
122  }
123  return nil
```

Table 2: Silent failures. All return success to a lenient client. Codes are the strict loop's, at `Next`, on every call.

Query / class	Win 10-based	Win 11-based
`SELECT Name FROM __EventConsumer`	0x80041017	0x80041017
`SELECT InstanceID, Program, Package FROM MSFT_Net ApplicationFilter`	0x80041017	0x80041017
`... FROM MSFT_Scheduled TaskInfo`	0x80041010	0x80041010
`Win32_SystemDriver` with any configuration property (or `*`)	333 of 334; `vwifibus` (binary absent) omitted	384 of 384 (no driver lacks its binary)

When `Next` fails with zero items, the loop condition is false, the error is never inspected and the function returns `nil`. The same gap truncates a result silently when `Next` fails after some items have been returned: those items are delivered, the terminating error is dropped, and the caller sees a short but "successful" enumeration. On both hosts, `SELECT * FROM` a non-existent class fails only at `Next` with `WBEM_E_INVALID_CLASS` (0x80041010); `ForEach` reports success with 0 items, while our strict loop records the error.

Production queries that never worked. Three of Agent A's 23 queries fail at `Next` on every call, with the same codes on both architectures (Table 2). Before the agent was stopped, WMI-Activity recorded 84 `5858` errors for each in 7 h, while the agent's audit log shows a 0-row, 0 ms success at each of their positions in every cycle (the log has no query text, so attribution is by position). From the agent's code, the consequences are concrete: firewall rules are shipped without their application path, non-standard event consumers are missing from the persistence inventory, and scheduled tasks lack last-run and next-run times. Each was a plausible-looking query. `Name` is not a property of the abstract `__EventConsumer` base class (which has `CreatorSID`, `MachineName` and `MaximumQueueSize`); it appears only on the five derived consumer classes [20]. `MSFT_NetApplicationFilter` exposes `AppPath` and `Package` but no `Program` (class definition read on-host with `Get-CimClass`); "Program" is the name of the `Set-NetFirewallApplicationFilter` cmdlet parameter [21], the likely origin of the mistake. `MSFT_ScheduledTaskInfo` is not a WMI class; the per-task run times are exposed through `MSFT_TaskDynamicInfo` (`LastRunTime`, `NextRunTime`, `LastTaskResult`, `NumberOfMissedRuns`; class definition read on-host with `Get-CimClass`) [22].

A provider that drops rows. On the Windows 10-based host `Win32_SystemDriver` returned 333 drivers whenever the query selected any configuration property (`DisplayName`, `PathName`, `StartMode`, `StartName`, `ServiceType`, `Description`, or `*`), and 334 for `SELECT Name` or `SELECT Name, State`. The missing driver, `vwifibus`, is registered with the service control manager but its image path points to a file that does not exist. No error is returned at any layer; only the comparison with the service control manager reveals it. A provider might reasonably omit a driver whose image cannot be read; what makes this a silent failure is that the row count depends on the projection—present for `SELECT Name`, absent once any configuration property is requested—with no error or other signal at any layer. On the Windows 11-based host the same driver is registered with its binary present in the driver store, and `Win32_SystemDriver` returned all 384 registered drivers for every selection. The omission therefore depends on host state, a registered driver whose binary is missing, which is common after driver removal, rather than on the Windows architecture. Because it is deterministic, we treat it as a per-key truth baseline and count S2 only above it.

Races are not failures. In 19,716 calibration queries the only other flags were three scheduled-task counts one short as the count changed and one process that exited during its query: timing, not WMI, separable only by bracketing truth before and after each query.

7 Fault recovery and refresh

If connections do not age, refresh is only worth its cost as recovery from faults. On each host we ran four 90-minute arms back to back ($k = k_H$, fresh connections each), one per strategy: *none* (E0); *timed*, redial every connection after 1 h (E1); *adaptive*, retire a connection when a failed call's effective error code is connection-level and redial at its next query (E2); and a replica of Agent A's *pool*, which recycles at 6 h, 50,000 queries or 5 min idle, or when the outer `HRESULT` maps to access denied, transport failure, "not available" or COM not initialised (E3; rules copied from the agent's source). At 15 min we killed the `WmiPrvSE` host serving `Win32_Process` and its co-hosted providers; at 30 min we restarted `Winmgmt` and its running dependent services through the service control manager (1.9 s in total on the Windows 10-based host, which had two running dependents; 0.2–0.4 s on the Windows 11-based host, which had none). The three deterministically failing keys are excluded.

A provider-host crash cost at most the call in flight. On the Windows 10-based host each of the four kills landed between queries and no query failed: the connection is to `Winmgmt`, which had a replacement host with the same providers running 31 ms later (E0), and the 99th-percentile latency in the following 30 s was unchanged (225 vs. 231 ms before). On the Windows 11-based host, where queries take longer, each kill landed while two calls served by the killed host were in flight, at the same point of the schedule in all four arms. A firewall-address query that had started 31–34 ms earlier and was still waiting for its first object failed about 10 ms after the kill, at `IEnumWbemClassObject::Next`,

Table 3: E-fault, 90 min per arm. Errors caused by the host kill and by the Winmgmt restart (the 60 min after it holds 60,000 and 30,600 queries at the two hosts' rates); a call in flight when a fault began is counted with that fault (Windows 11-based host only: one call at each kill, two or three at each restart); recovery is the time until every connection that failed has answered again.

		Windows 10-based, $k = 100$			Windows 11-based, $k = 50$		
Arm	Strategy	kill errors	restart errors	recovery	kill errors	restart errors	recovery
E0	none	0	59,995	none in 60 min	1	30,605	none in 60 min
E1	timed (1 h)	0	30,002	1,804.5 s	1	15,311	1,811.5 s
E2	adaptive	0	8	6.0 s	1	10	11.5 s
E3	Agent A pool	0	59,995	none in 60 min	1	30,602	none in 60 min

with `RPC_S_CALL_FAILED` (0x800706BE); this agrees with Microsoft's guidance that when WMI stops a host for a quota violation the query it is serving fails [14]. A driver query that started about 245 ms earlier and was already returning rows completed with all 384 drivers, matching the service control manager. In both cases the connection was unaffected: under no refresh, the timer and the pool the next query on it succeeded and no other query failed before the service restart 15 min later, and the 99th-percentile latency was unchanged (687 vs. 692 ms before, E0). The adaptive client treats `RPC_S_CALL_FAILED` as connection-level, so it retired the healthy connection and redialled in 7.8 ms. The failure surfaces at `Next`, the call whose result the `ForEach` helper of Section 6 discards, so a client using it would have recorded an empty result as a success. These are four kills at one offset into a call, not a map of every phase of a call. **[a kill via a lowered `MemoryPerHost` quota was not run; drop or schedule]**

A service restart is permanent. After `Winmgmt` restarted, every call on every existing connection failed with `RPC_S_SERVER_UNAVAILABLE` (0x800706BA) and none recovered on its own (Windows 10-based E0: 59,995 of 60,000; the 5 successes were calls issued in the 175 ms before the service stopped; Windows 11-based E0: all 30,605, including three calls in flight when the restart began, which failed with `RPC_E_DISCONNECTED` (0x80010108) or `RPC_S_CALL_FAILED`). The timed strategy was down until its timer fired, 30 minutes later; an outage of up to one refresh interval is inherent to it. This is not peculiar to Agent A: Microsoft documents that its Configuration Manager client (`CcmExec`) is left nonfunctional after the WMI service is restarted, because WMI cancels outstanding requests and does not re-establish them, and recovers only when a periodic health check restarts the client [13]. The adaptive client lost exactly one query per connection: each of its 9 connections on the Windows 10-based host failed once, was retired, redialled in 1–6 ms, and its next scheduled query succeeded; Table 3 counts 8 because one of the nine failures fell on an excluded deterministic key. On the Windows 11-based host all 10 connections failed once (two on calls in flight when the restart began) and redialled in 1–36 ms. The recovery time, 6.0 and 11.5 s, is the query interval of the least-busy namespace at each host's k_H and would be one production polling interval at $k = 1$. Agent A's pool behaved exactly like no refresh on both hosts (59,995 and 30,602 errors, no recycle): `RPC_S_SERVER_UNAVAILABLE` is not an outer code its health check treats as unhealthy, and at arm end its connections were 1.5 h of 6 h old with at most 21,000 (10,500 on the Windows 11-based host) of 50,000 queries. By its rules the first recycle would come from the query budget at about 214 min at $k = 100$, from the 6 h age limit at $k = 50$ and at the production rate; we did not run that long. The refresh mechanism did not fire on the one fault that needed it, on either host.

8 Discussion

Guidance. (1) Make the client correct first: one STA per connection, all COM calls on it, every `VARIANT` cleared, and a test that runs a few hundred thousand queries and checks for growth; timed recycling hides neither of the bugs above. (2) Never let an enumeration end without inspecting the last `HRESULT`; audit libraries for this pattern. (3) Validate each query once at start-up and verify results against an independent source where one exists; "0 rows" is a claim, not a fact. (4) Reconnect on connection-level errors by inspecting the effective error code, not only the outer `HRESULT`; test the health check by restarting `Winmgmt`. In our fault injection the service restart produced `RPC_S_SERVER_UNAVAILABLE` (observed); a robust health check should additionally treat `WBEM_E_TRANSPORT_FAILURE` and `WBEM_E_NOT_AVAILABLE` as connection-level (documented [17], not observed here), and `RPC_E_DISCONNECTED` (observed on calls in flight when the service stopped). `RPC_S_CALL_FAILED` from a crashed provider host left the connection usable; retiring on it costs one redial. A timer adds churn and leaves an outage of up to one interval. (5) Do not attribute `WmiPrvSE` growth to your client without per-host attribution and a foreign-client meter.

Threats to validity. Two hosts, one per architecture, and one workload; each fault was injected once per strategy and host, so the fault results are existence proofs of a strategy's behaviour, not rates. The two hosts also differ in acceleration (the Windows 10-based ran at $k = 100$, the Windows 11-based at $k = 50$, forced by the 40% CPU budget on its slower virtual CPU model), in RAM and in CPU model, so architecture, rate and hardware move together; the between-host difference in client memory growth cannot be attributed to the operating system alone. Accelerating the query rate

tests count, not age, and not the provider unload/reload cycle, which at k_H does not occur: our longest correct-client connections are 6 h old; the 7-day run is in progress on the Windows 10-based host only. On the Windows 11-based host the client's connection-held memory grew throughout the 6 h arm; whether it levels off is open. The ETW meter ran one minute in fifteen during the fault arms, so foreign activity there is sampled (on the Windows 10-based host it saw one burst of 6 operations, most likely the Windows Update service; on the Windows 11-based host 82 operations in two arms from a system service polling network-adapter classes); it ran continuously during the self-test and client-bug arms. Agent A's production numbers come from before/after comparisons of single changes on a few hosts. Our no-degradation result is for a client whose COM usage is correct on this provider set; providers themselves can leak, as in a documented `WmiPrvSE` user-profile case [15], which per-host attribution (guidance 5) is designed to separate from client growth. Older and server-only architectures, and further client builds, remain future work.

Related work. Microsoft documents the mechanisms we rely on: provider hosting and unloading [1, 2], quotas [4], enumeration semantics [5] and COM apartment rules [6]. The newer Management Infrastructure (MI) API [16] is the client surface the CIM cmdlets use and a natural target for replication; our findings here are for a COM/scripting client, and whether they transfer to MI and to native `IWbemServices` clients is open. We are not aware of published measurements of WMI connection ageing under a verified-correct client, nor of a catalogue of WMI silent failures; we welcome pointers.

9 Conclusion

Long-lived WMI connections, used correctly, did not degrade in latency, median or tail, on either architecture; the provider hosts and the WMI service grew by less than 2 MB over the 5.5-hour measurement window, mostly in its first third, and the client by at most 1.5 MB, all of it returned when the connections closed; the symptoms we could reproduce were produced by the client, and the costliest failures were silent. Refresh is a recovery mechanism, best driven by errors. The probe, catalogue and analysis code are available from the authors **[replace with a public repository URL and an artifact DOI]**.

References

[1] Microsoft. *Provider Hosting and Security*. https://learn.microsoft.com/en-us/windows/win32/wmisdk/provider-hosting-and-security

[2] Microsoft. *Unloading a Provider*. https://learn.microsoft.com/en-us/windows/win32/wmisdk/unloading-a-provider

[3] Microsoft. *__CacheControl class*. https://learn.microsoft.com/en-us/windows/win32/wmisdk/--cachecontrol

[4] Microsoft. *__ProviderHostQuotaConfiguration class*. https://learn.microsoft.com/en-us/windows/win32/wmisdk/--providerhostquotaconfiguration

[5] Microsoft. *IEnumWbemClassObject::Next*. https://learn.microsoft.com/en-us/windows/win32/api/wbemcli/nf-wbemcli-ienumwbemclassobject-next

[6] Microsoft. *Single-Threaded Apartments*. https://learn.microsoft.com/en-us/windows/win32/com/single-threaded-apartments

[7] Microsoft. *VariantClear function*. https://learn.microsoft.com/en-us/windows/win32/api/oleauto/nf-oleauto-variantclear

[8] Microsoft. *Tracing WMI Activity*. https://learn.microsoft.com/en-us/windows/win32/wmisdk/tracing-wmi-activity

[9] go-ole contributors. *go-ole v1.3.0, oleutil/oleutil.go*. https://github.com/go-ole/go-ole/blob/v1.3.0/oleutil/oleutil.go

[10] W. K. Newey and K. D. West. A simple, positive semi-definite, heteroskedasticity and autocorrelation consistent covariance matrix. *Econometrica*, 55(3):703–708, 1987.

[11] StackExchange. *wmi: Go WMI package*, README warning on the memory bug. https://github.com/StackExchange/wmi

[12] Zabbix. *ZBX-8506: wmi.get on Windows agent causes memory leak (unreleased pclsObj)*. https://support.zabbix.com/browse/ZBX-8506

[13] Microsoft. *SMS Agent Host service isn't automatically restarted after the WMI service restarts*. https://learn.microsoft.com/en-us/troubleshoot/mem/configmgr/client-management/sms-agent-host-not-automatically-restarted

[14] Microsoft. *Scenario guide: troubleshoot WmiPrvSE quota-exceeded issues*. https://learn.microsoft.com/en-us/troubleshoot/windows-server/system-management-components/scenario-guide-troubleshoot-wmiprvse-quota-exceeded-issues

[15] Microsoft. *Memory leak occurs in the Wmiprvse.exe process in Windows 7 or Windows Server 2008 R2* (KB2832248). https://support.microsoft.com/en-us/topic/067c6067-0150-ce7e-3c8a-ba762a0b1168

[16] Microsoft. *Management Infrastructure (MI) reference*. https://learn.microsoft.com/en-us/windows/win32/api/mi/

[17] Microsoft. *WMI Error Constants*. https://learn.microsoft.com/en-us/windows/win32/wmisdk/wmi-error-constants

[18] Microsoft. *IEnumVARIANT::Next method*. https://learn.microsoft.com/en-us/windows/win32/api/oaidl/nf-oaidl-ienumvariant-next

[19] Microsoft. *Service Host grouping in Windows 10*. https://learn.microsoft.com/en-us/windows/application-management/svchost-service-refactoring

[20] Microsoft. *__EventConsumer class*. https://learn.microsoft.com/en-us/windows/win32/wmisdk/--eventconsumer

[21] Microsoft. *Set-NetFirewallApplicationFilter*. https://learn.microsoft.com/en-us/powershell/module/netsecurity/set-netfirewallapplicationfilter

[22] Microsoft. *Get-ScheduledTaskInfo* (outputs `MSFT_TaskDynamicInfo`). https://learn.microsoft.com/en-us/powershell/module/scheduledtasks/get-scheduledtaskinfo